What IF I had No Smells?

Davide Falessi

Agenda

- Paper
  - Introduction
  - Metric selection,
  - Metric collection,
  - Dataset manipulation,
  - Model selection, and
  - Estimation results.

INTRODUCTION

- Code bad smells, or shortly smells, are code artifacts that are symptoms of poor design and implementation choices [1] [2] [3].
- There are several smell types [1], such as large classes (aka God classes), code clones, low comment frequency, and non-intuitive variable naming.
- Smells can leave in the code some "technical debt" (i.e., not-quite good code left in the released software) that can eventually lower the quality of the product.

Aim

- Describe a method to estimate the number of defective files (i.e., files with at least one defect) if smells were avoided.

What-if Analysis Method

- Our method is based on a What-if analysis.
- We define a What-if scenario (i.e., if smells are avoided), prepare data accordingly, learn classifiers on actual data, and estimate file defectiveness on synthetic data obtained if the code had no smells.

Context

- Our method is applied on data collected from five different industrial projects and 106 different types of smells of one company called Keymind.
- To increase the generalization power of our work, we also provide a set of assumptions under which our method can be replicated on different data
- The method consists of five steps…

Metric Selection

- Besides the initial size and number of smells and the final number of defective files, we selected 16 change metrics that have been extensively used for defect estimation [29].

Metric (File C)	Description
Size	LOC
LOC_touched	Sum over revisions of LOC added+deleted+modified
NR	Number of revisions
NFix	Number of bug fixes
NAuth	Number of Authors
LOC_added	Sum over revisions of LOC added
MAX_LOC_added	Maximum over revisions of LOC added
AVG_ LOC_added	Average LOC added per revision
Churn	Sum over revisions of added – deleted LOC in C
MAX_Churn	MAX_CHURN over revisions
AVG_Churn	Average CHURN per revision
ChgSetSize	Number of files committed together with C
MAX_ChgSet	Maximum of ChgSet_Size over revisions
AVG_ChgSet	Average of ChgSet_Size over revisions
Age	Age of C in weeks
WeightedAge	Age weighted by LOC_touched
NSmells	Number of smells

METRIC COLLECTION

- With SonarQube, which is able to identify 1297 different smell types, we collected the number of smells in a file per single revision. For each release and file, we computed the size and the number of smells just before the first revision and the change metrics and defects at the last revision.

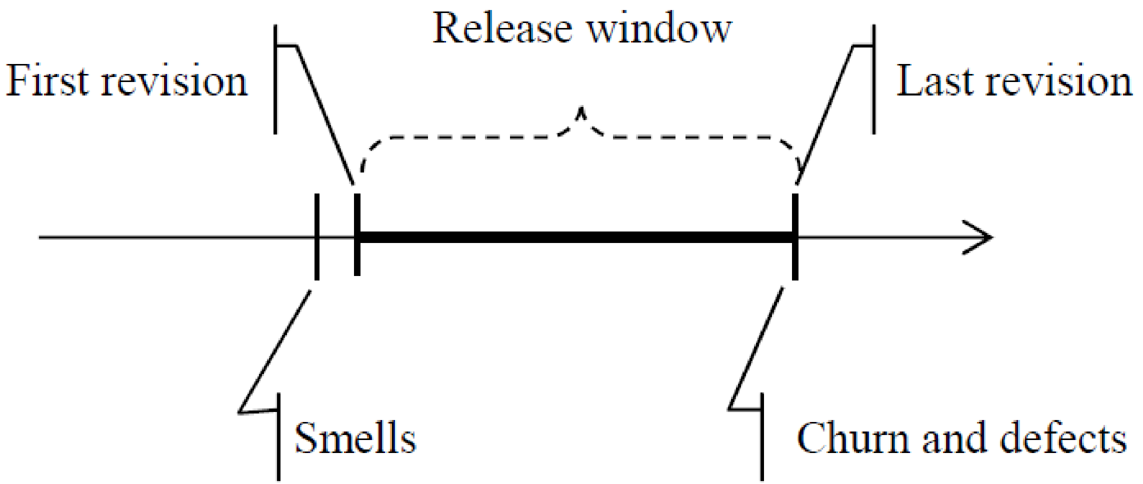

Figure 2: Data collection per release and file

Dataset Manipulation

- A: Original dataset
- B+: Portion of A with #smells >0
- C portion of A with #smells =0.
- B: B+ manipulated with feature #smells set to 0

Dataset Manipulation

- We measured for each feature the average value and the correlation with NSmells and Defectiveness

	Mean			Correlation (ρ)	
Variable	**A**	**B**	**C**	**NSmells**	**Defectiveness**
Size	289	271	303	0.31*	0.18*
LOC_touched	57	49	60	-0.01	0.25*
NR	9	8	10	0.02	0.38*
NFix	3	2	3	0.02	0.51*
NAuth	0.7	0.7	0.7	-0.07	0.24*
LOC_added	31	21	39	-0.04	0.18*
MAX_LOC_added	21	15	26	-0.04	0.18*
AVG_ LOC_added	12	9	14	-0.03	0.14*
Churn	16	14	17	0.01	0.12*
MAX_Churn	10	9	11	0.02	0.10*
AVG_Churn	7	7	6	0.02	0.09*
ChgSetSize	6	6	6	-0.06	0.29*
MAX_ChgSet	3	3	3	-0.05	0.19*
AVG_ChgSet	2	2	2	-0.04	0.06
Age	40	42	38	0.18*	0.02
WeightedAge	11	13	10	0.06	0.10*
NSmell	12	0	0	-	0.10*

Model Selection

- Four models (Bagging, BayesNet, J48, and Logistic) were evaluated using leave-one-out cross-validation. Bagging outperformed the others.

	Bagging	BayesNet	J48	Logistic
Precision	0.79	0.62	0.72	0.73
Recall	0.70	0.52	0.70	0.51

Results

- We applied the trained Bagging model on the dataset A, B+, B, and C.
- The drop is a substantial 42% ((66-38)/66). This means an overall reduction of 20% ((66-38)/135) in the number of defective files on the whole dataset A.

Dataset A		Dataset B+		Dataset B	Dataset C	
A	E	A	E	E	A	E
135	120	66	57	38	69	63